

Frontend – Theory Assignment 1

Q1. HTTP Request–Response Cycle

Explain the HTTP Request–Response Cycle with a suitable diagram. Describe the major steps involved when a client (browser) requests a resource from a web server and receives a response.

Answer: The HTTP request–response cycle describes how a browser and a web server exchange information whenever a page is loaded. The browser (client) sends out a request for a resource, and the server replies with the requested content or an error message.

The cycle generally unfolds in the following stages:

1. The user types a URL in the address bar or clicks on a hyperlink.
2. The browser translates this action into an HTTP request, specifying the resource and the method (usually GET or POST).
3. This request travels over the network to reach the destination web server.
4. The server interprets the request and locates or generates the requested resource.
5. The server sends back an HTTP response, which may contain HTML markup, CSS, JavaScript, images, or JSON data, along with a status code.
6. The browser reads the response and renders the page (or handles the returned data) for the user to view.

[Diagram: Browser --Request--> Server --Response--> Browser]

Q2. Introduction to Web Technologies

Briefly explain the purpose and key features of HTML, CSS, JavaScript, Bootstrap, jQuery, and React.js.

HTML: HyperText Markup Language provides the skeleton of a webpage. Using tags such as headings, paragraphs, images, links, and tables, it defines what content appears on the page and how it is structured.

CSS: Cascading Style Sheets is responsible for the visual presentation of a page — things like colour schemes, fonts, spacing, borders, and overall layout.

JavaScript: JavaScript brings interactivity to a page. It can respond to clicks, validate form entries, update content dynamically, and communicate with servers without reloading the page.

Bootstrap: A popular CSS framework that ships with pre-built classes and UI components, letting developers build responsive, good-looking interfaces without writing every style rule from scratch.

jQuery: A lightweight JavaScript library that simplified common tasks — DOM manipulation, event handling, AJAX calls, and animations — with much shorter code than plain JavaScript required at the time.

React.js: A JavaScript library from Meta for building interactive UIs out of small, reusable components. It efficiently re-renders only the parts of the page that change, which makes complex interfaces easier to manage.

Q3. Client-Side vs. Server-Side Technologies

What are client-side technologies and server-side technologies? Explain the difference between them with suitable examples.

Answer: Client-side technologies execute inside the user's browser and are responsible for what the visitor actually sees and interacts with. HTML, CSS, and JavaScript fall into this category.

Answer: Server-side technologies, on the other hand, run on the web server. They handle tasks such as processing requests, querying databases, applying business logic, and preparing the data that is eventually sent to the browser. Common examples include Node.js, PHP, Java, and Python frameworks like Flask or Django.

In short, client-side code shapes the experience the user sees in real time, while server-side code manages data and logic behind the scenes before anything reaches the browser.

Q4. Block-Level Elements in HTML

What are block-level elements in HTML? Explain their characteristics and provide at least three examples.

Answer: Block-level elements are HTML elements that occupy the full width available to them and always begin on a new line, effectively creating distinct blocks of content on a page.

Key characteristics:

- Each one starts on a fresh line rather than flowing inline with surrounding text.
- By default, it stretches to fill the width of its parent container.
- Multiple block-level elements stack vertically, one after another.
- They are commonly used to structure a page into logical sections.

Examples: <div>, <p>, <h1>, <section>, <header>.

Q5. Semantic HTML Elements

What are semantic elements in HTML? Explain their importance in web development and provide suitable examples.

Answer: Semantic elements are tags whose names describe the meaning or role of the content they wrap, rather than just how it should appear visually — for example, using <nav> for navigation links instead of a generic <div>.

Why they matter: semantic markup makes the underlying code more readable and easier to maintain, helps search engines understand and index page content more accurately, and improves accessibility for screen readers and other assistive tools.

Examples: <header>, <nav>, <main>, <section>, <article>, <aside>, <footer>.

Q6. Creating a Table Using HTML

Write HTML code to create the given table with Sr. No., Name, Designation, and Department columns.

```
<table border="1">
  <tr>
    <th>Sr. No.</th>
    <th>Name</th>
```

```

    <th>Designation</th>
    <th>Department</th>
</tr>
<tr>
    <td>1</td>
    <td>John</td>
    <td>Manager</td>
    <td>Sales</td>
</tr>
<tr>
    <td>2</td>
    <td>Smith</td>
    <td>Sr. Manager</td>
    <td>Marketing</td>
</tr>
<tr>
    <td>3</td>
    <td>Jane</td>
    <td>Manager</td>
    <td>HR</td>
</tr>
</table>

```

Rendered output:

Sr. No.	Name	Designation	Department
1	John	Manager	Sales
2	Smith	Sr. Manager	Marketing
3	Jane	Manager	HR

Q7. Creating a Nested List Using HTML

Write HTML code to create the nested list using an appropriate combination of ordered/unordered lists.

```

<ul>
    <li>River
        <ul>
            <li>Ganga</li>
            <li>Yamuna</li>
            <li>Brahmaputra</li>
            <li>Narmada</li>
        </ul>
    </li>
</ul>

```

Q8. Types of CSS

Explain the different types of CSS used for applying styles to an HTML document. Describe each type with a suitable example.

Answer: CSS can be applied to a page in three ways:

1. Inline CSS — styling is written directly inside a tag's style attribute. It only affects that single element and is handy for quick, one-off changes.

```
<p style="color: blue;">Hello</p>
```

2. Internal CSS — rules are placed inside a <style> block within the document's <head>. This works well when the styling only needs to apply within that one page.

```
<style>
```

```
  p { color: blue; }
```

```
</style>
```

3. External CSS — styles live in a separate .css file that is linked to the HTML page. This approach is preferred when the same styling needs to be reused across multiple pages.

```
<link rel="stylesheet" href="style.css">
```

Q9. Practical Understanding

Differentiate between HTML, CSS, and JavaScript based on their role in developing a web page. Explain what each technology contributes to a website.

Answer: These three technologies each handle a distinct layer of a website:

- HTML builds the structure and content — the headings, paragraphs, images, and other elements that make up the page.
- CSS controls the presentation layer — colours, fonts, spacing, and overall layout — turning plain structure into a visually styled page.
- JavaScript adds behaviour and interactivity, allowing the page to respond to user actions, update content dynamically, and communicate with servers.

Together, HTML provides the 'what', CSS provides the 'how it looks', and JavaScript provides the 'how it behaves'.

Q10. Short Answer

a. What is the role of Bootstrap in web development?

Answer: Bootstrap supplies a library of pre-styled CSS classes and ready-made UI components, letting developers quickly assemble responsive, professional-looking pages without writing extensive custom CSS.

b. What is jQuery and why was it widely used in web development?

Answer: jQuery is a JavaScript library that wraps common browser tasks — selecting elements, handling events, running animations, and making AJAX requests — into simple, concise method calls. It became popular because it reduced the amount of code needed and smoothed over inconsistencies between different browsers.

c. What is React.js and what problem does it solve in frontend development?

Answer: React.js is a JavaScript library for constructing user interfaces from reusable components. It solves the problem of efficiently updating the UI as data changes, using a virtual DOM to re-render only the parts of the page that actually need to change rather than the whole page.